

Problem Statement

Most individuals struggle with **financial planning and budgeting** because their financial data is:

- **Fragmented** across bank apps, wallets, and statements
- Difficult to **analyze manually**
- Lacking **intelligent insights**

As a result, users:

- Cannot track spending effectively
- Fail to maintain budgets
- Miss opportunities to save

Goal: Build an AI-powered platform that **automatically tracks, categorizes, and analyzes financial data**, providing actionable insights

->NAVNEET SINGH

(2301020147)

MEMBER 4

DAY1

Data Ingestion (Plaid API / CSV Upload)



Transaction Storage (Database)



Categorization Engine (LLM)



LangChain ReAct Agent (AI reasoning & tool use)



Streamlit UI (dashboard + insights)

TASK1- `pip install scikit-learn pandas openai langchain`
(VERIFIED WITH NO ERRORS)

TASK 2- Write agent/tools.py

Write 4 function stubs with docstrings only: `get_transactions(user_id, start_date, end_date)`, `get_budget_status(user_id, month)`, `calculate_savings_projection(monthly_savings, months, annual_rate)`, `categorize_spending(description)`. Add a comment above each explaining what it will do on Day 3.

TASK 3-Transaction Categorization Engine

Add "Transaction Categorization Engine" section. Explain: use few-shot LLM prompting to classify raw text. Show 3 examples: "SWIGGY ORDER 4421" → Food, "UBER TRIP 8823" → Transport, "BESCOM BILL" → Housing

Transaction Categorization Engine

The **Transaction Categorization Engine** is responsible for automatically classifying raw financial transaction descriptions into meaningful spending categories, helping users understand their spending patterns without manual effort.

Objective

Financial transaction data often comes in **unstructured text format** (e.g., merchant names, codes, or abbreviations), making it difficult to track expenses or budget effectively. This engine leverages AI to convert raw text into structured categories.

Approach: Few-Shot LLM Prompting

Instead of relying on rigid keyword rules, we use a **few-shot learning approach with a Large Language Model (LLM)**. The model is given a few labeled examples and generalizes to classify unseen transaction descriptions correctly.

Few-Shot Prompt Example

The LLM receives:

Instruction: "Categorize the following transaction into one of these categories: Food, Transport, Housing, Entertainment, Others."

Examples:

- "SWIGGY ORDER 4421" → Food
- "UBER TRIP 8823" → Transport
- "BESCOM BILL" → Housing

New input: "DOMINOS PIZZA 324"

Model output: Food

How It Works

1. **Input:** User transaction (from bank, wallet, or CSV)
2. **Processing:** LLM classifies the description using few-shot examples
3. **Output:** Category label stored in the database and used for analytics

DAY2

TASK1 – Build the categorization function

Build the categorization function

In agent/categorize.py, write `categorize_transaction(description)` using a few-shot OpenAI prompt. Pass 6 examples in the prompt, then the new description. Return the category label.

TASK -2

Test with 10 sample inputs

Test: "SWIGGY ORDER"→Food, "UBER TRIP"→Transport, "BESCOM BILL"→Housing,

"NETFLIX"→Entertainment, "AMAZON"→Shopping, "ATM WITHDRAWAL"→Other. Print results and confirm all 10 pass

TASK 3–

Connect categorizer to ingestion

Update database/ingest.py: for rows where category is blank, call `categorize_transaction(description)` automatically and fill it in before inserting to DB

TASK 4–

PUSH TO THE GITHUB